JBoss Community / Debezium: Milvus Source Connector for Debezium

Note: Make sure to include the sub-org name in the title both in Google's system and in your document.

About Me

- **Name:** Yuang Li (Aiden Li)
- **University:** Northeastern University
- **Program:** M.S. in Computer Science
- **Expected Graduation:** December 2026
- **Email:** Yuangli971@gmail.com
- **Time Zone:** America/New_York(Eastern Standard Time)

Code Contribution

- Contribution.
  - add pravega connection validator
  - add Infinispan connection validator
  - Start & Stop API for the Quarkus Runtime Extension

Project Information

Abstract

Milvus is a cloud-native vector database for high-performance similarity search. Debezium currently provides a Milvus sink connector, but not a source connector, so changes within Milvus cannot yet be captured for replication, auditing, or downstream processing.

This project proposes a Debezium source connector for Milvus 2.5, covering initial snapshot, streaming CDC, schema handling, offset management, and restart recovery. Kafka is the primary target, with Pulsar support considered as a stretch goal.

Milvus 2.5 is selected because its Kafka/Pulsar-based architecture is comparatively stable and well understood. Milvus 2.6 introduces Woodpecker and a new StreamingService layer, which significantly changes the consumption model and still appears to be evolving, so support for 2.6 is deferred as future work.

Why this project?

During a previous internship I worked with Kafka-based data pipelines, which led me to CDC and how it keeps distributed systems consistent. A big part of that was reading Jay Kreps The Log, the idea that an append-only log is the unifying abstraction behind databases, replication, and stream processing changed how I think about distributed systems.

What I liked most was that building this connector is essentially putting theories that I learned into practice, using Milvus's internal WAL and message queue as the source of truth, and building a reliable change stream on top of it. Figuring out why etcd and the message queue need to work together, finding the timetick mechanism for DDL/DML ordering, tracing how milvus-cdc uses etcd checkpoints as MQ seek positions. These were problems I enjoyed digging into, and they felt like a direct application of the ideas that got me interested in this space in the first place.

Technical Description

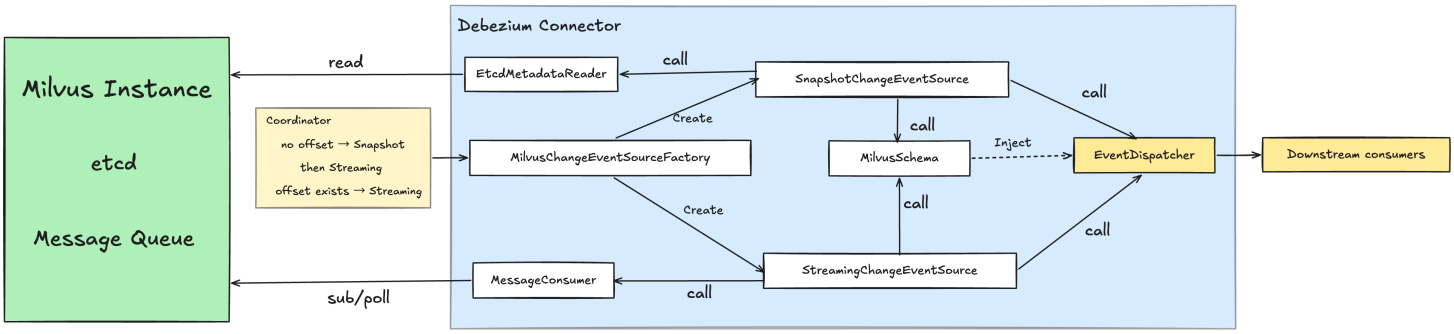
Scope

Primary goal: Implement a fully working Debezium source connector for Milvus 2.5 with Kafka as the message queue backend — covering initial snapshot, streaming CDC, schema handling, offset management, and restart recovery.

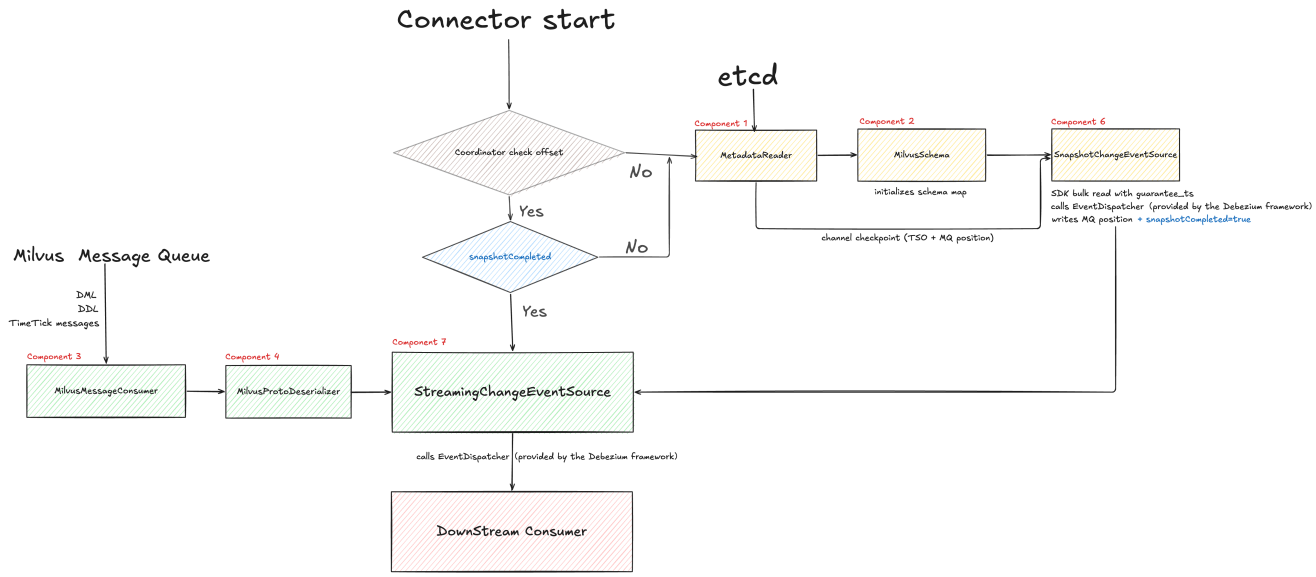
Stretch goal: Extend `MilvusMessageConsumer` with a Pulsar implementation. Since the interface is already abstracted, this is an incremental addition that does not affect the core pipeline design.

Future work: Woodpecker WAL support for Milvus 2.6, pending stabilization of WAL DDL support.

Technical Architecture



Data Flow



The flow diagram above covers the following cases:

- **Case 1**: First start (no offset) — full snapshot then streaming
- **Case 2**: Restart, schema unchanged — resume streaming from last MQ position
- **Case 3**: Restart, schema changed — resume streaming, schema updated via `CreateCollectionMsg` in MQ
- **Case 4**: Snapshot interrupted — `snapshotCompleted=false` detected on restart, full snapshot re-executed
- Additional Edge Cases see in Appendix

Technical Challenges

Snapshot-to-streaming handoff consistency

Challenge:

Milvus TSO timestamps and MQ offsets are two different coordinate systems. A connector must ensure that the initial snapshot and the streaming phase meet at the same logical point, without gaps or duplicates.

Approach:

use the channel checkpoint stored in etcd as the anchor.
Before the bulk read, read the MQ seek position from etcd and extract the embedded TSO as `guarantee_ts` for the SDK query(Milvus's Strong consistency level), Start streaming from exactly that MQ position after snapshot completes.

Milvus's Strong consistency level:
when a query is issued with a guarantee_ts,the QueryNode blocks until all writes before that timestamp are visible, giving the snapshot a well-defined point-in-time view.

Key Components

Component 1: EtcdMetadataReader

Responsibility:

- Connects to etcd to read collection schema, vchannel names, and the current MQ channel checkpoint position.

```java
public class EtcdMetadataReader {

public EtcdMetadataReader(MilvusConnectorConfig config);

/** Reads field definitions for a given collection */
public CollectionSchema readSchema(String collectionName)
        throws DebeziumException;

/** Reads vchannel → topic mappings for all configured collections */
public Map<String, String> readVchannelTopics(List<String> collections);

/** Reads the current MQ channel checkpoint position from etcd. */
public MilvusChannelPosition readChannelCheckpoint(String vchannel);

}
```

Implementation notes:
The channel checkpoint stored in etcd is a Protobuf-encoded MQ seek position containing an embedded TSO timestamp. This is the anchor used to align snapshot and streaming (see details in Technical Challenge 1).

Component 2: MilvusSchema

Responsibility:

- Implements Debezium's `DatabaseSchema<CollectionId>` interface to maintain the current Debezium schema state for all monitored collections.
- Converts Milvus `CollectionSchema` (Protobuf format) into Debezium `Schema` (Kafka Connect format) internally.
- Serves as the single source of truth for collection schemas, injected into `EventDispatcher` at construction time.

```java
public class MilvusSchema implements DatabaseSchema<CollectionId> {

    public MilvusSchema(MilvusConnectorConfig config);

    /** Called by EventDispatcher when building INSERT/DELETE change events */
    @Override
    public DataCollectionSchema schemaFor(CollectionId collectionId);

    /** Builds and registers the Debezium schema for a collection from its Milvus Colle
    public void applySchemaChange(CollectionId collectionId, CollectionSchema milvusSch

    /** Removes the schema entry for a dropped collection */
    public void removeSchema(CollectionId collectionId);
}
```

Implementation notes:

This conversion is used both at startup, when schema metadata is loaded from etcd, and during streaming, when a `CreateCollectionEvent` updates the in-memory schema state. Schema change events are emitted before subsequent DML events for that collection are processed.

Component 3: MilvusMessageConsumer (abstract interface)

Responsibility:

- Abstracts the difference between Kafka and Pulsar,
- WAL(version 2.6) in the future

```java
public interface MilvusMessageConsumer extends AutoCloseable {

    /** Subscribes to the topic for a given vchannel, starting from the specified offse
    void subscribe(String vchannel, MilvusOffset fromOffset);

    /** Polls a batch of raw messages */
    List<RawMessage> poll(Duration timeout);

    /** Commits the current consumption position */
    void commit(MilvusOffset offset);
}

// Two implementations — selected at startup based on mq.type config
public class KafkaMilvusMessageConsumer implements MilvusMessageConsumer {}
public class PulsarMilvusMessageConsumer implements MilvusMessageConsumer {}
```

Kafka and pulsar share the same message body format: `milvus-proto Protobuf` :

- The Kafka implementation uses `kafka-clients` with offset as `topic + partition + long` .
- The Pulsar implementation uses `pulsar-client` with offset as `MessageId` (`ledgerId + entryId`)

Component 4: MilvusProtoDeserializer

Responsibility:

- Parses raw MQ payloads into structured Milvus change events.
- Converts `column-oriented` INSERT payloads into `row-oriented` records for Debezium event emission.
- Extracts schema information from `collection-creation` messages for schema updates and schema change events.
- Extracts `timetick` watermarks used to order and flush buffered DDL and DML events.

```java
public class MilvusProtoDeserializer {

    public MilvusChangeEvent deserialize(RawMessage message)
        throws DebeziumException;
}

public sealed interface MilvusChangeEvent {
    record InsertEvent(String collection, List<Map<String, Object>> rows)
        implements MilvusChangeEvent {}
    record DeleteEvent(String collection, List<Object> primaryKeys)
        implements MilvusChangeEvent {}
    record CreateCollectionEvent(String collection, CollectionSchema schema)
        implements MilvusChangeEvent {}
    record DropCollectionEvent(String collection)
        implements MilvusChangeEvent {}
    record TimeTickEvent(long tso)
        implements MilvusChangeEvent {}
}
```

References

- Milvus `InsertRequest` definition in `msg.proto` : msg.proto
- Milvus `FieldData` definition in `schema.proto` : schema.proto
- Milvus internal column-to-row conversion: utils.go

Component 5:MilvusChangeEventSourceFactory

Responsibility:

- Implements Debezium's `ChangeEventSourceFactory` , called by `ChangeEventSourceCoordinator` at startup to construct the snapshot and streaming sources.

```java
import java.util.Optional;

public class MilvusChangeEventSourceFactory
        implements ChangeEventSourceFactory<MilvusPartition, MilvusOffsetContext> {

  @Override
  public SnapshotChangeEventSource<MilvusPartition, MilvusOffsetContext>
  getSnapshotChangeEventSource(
          SnapshotProgressListener<MilvusPartition> snapshotProgressListener,
          NotificationService<MilvusPartition, MilvusOffsetContext> notificationService
  );

  @Override
  public StreamingChangeEventSource<MilvusPartition, MilvusOffsetContext>
  getStreamingChangeEventSource();

  @Override
  public Optional<IncrementalSnapshotChangeEventSource<MilvusPartition, ? extends DataC
  getIncrementalSnapshotChangeEventSource(
          MilvusOffsetContext offsetContext,
          SnapshotProgressListener<MilvusPartition> snapshotProgressListener,
          DataChangeEventListener<MilvusPartition> dataChangeEventListener,
          NotificationService<MilvusPartition, MilvusOffsetContext> notificationService
  ) {
    return Optional.empty();
  }
}
```

- `getIncrementalSnapshotChangeEventSource` — returns `Optional.empty()` for now.
- Incremental snapshot is deferred because it requires a signaling mechanism and a chunking strategy compatible with Milvus's query API. The current abstraction in MilvusSnapshotChangeEventSource is designed to be reusable for this purpose once the core pipeline is stable.

Component 6:MilvusSnapshotChangeEventSource

Responsibility:

- Extends `AbstractSnapshotChangeEventSource` to perform the initial bulk read of existing collection data.

- Implements the **snapshot-to-streaming handoff** by reading the etcd channel checkpoint before snapshot execution.
- Uses the embedded TSO as `guarantee_ts` for a `consistency_level=Strong` Milvus query, ensuring that the snapshot reflects a well-defined point in time.
- Stores the corresponding MQ position in `MilvusOffsetContext` so streaming can continue from the matching position after snapshot completion or restart.

```
public class MilvusSnapshotChangeEventSource
        extends AbstractSnapshotChangeEventSource<MilvusPartition, MilvusOffsetContext>

  @Override
  protected SnapshotResult<MilvusOffsetContext> doExecute(
          ChangeEventSourceContext context,
          MilvusOffsetContext previousOffset,
          SnapshotContext<MilvusPartition, MilvusOffsetContext> snapshotContext,
          SnapshottingTask snapshottingTask
  ) throws Exception;
}
```

Component 7: MilvusStreamingChangeEventSource

Responsibility:

- Implements `StreamingChangeEventSource` to consume the message queue from the offset recorded at the end of the snapshot.
- Handles DML, DDL, and timetick messages emitted through Milvus MQ.
- Uses timetick watermarks to order and flush buffered schema changes and data changes before they are emitted through Debezium's event pipeline.
- Supports restart recovery by resuming consumption from the stored MQ position.

```java
public class MilvusStreamingChangeEventSource
        implements StreamingChangeEventSource<MilvusPartition, MilvusOffsetContext> {

  @Override
  public void execute(
          ChangeEventSourceContext context,
          MilvusPartition partition,
          MilvusOffsetContext offsetContext
  ) throws InterruptedException;
}
```

Testing Strategy

- Unit tests: cover isolated logic: deserialization, schema conversion, and offset serialization
- Integration tests: use Testcontainers to validate etcd reads, MQ consumption, snapshot, and streaming against real infrastructure
- End-to-end tests: validate full connector lifecycle: cold start, restart recovery, schema evolution, and edge cases (timetick stall, offset expiry, empty collection)

Roadmap

Community Bonding (May 1 – May 24)

- Confirm technical direction with mentors (etcd bootstrap, MQ consumption, timetick ordering)
- Set up local development and test environment
- Review `milvus-proto` message definitions and validate assumptions around TSO encoding and Strong consistency
- Prepare detailed implementation and testing plan and share with mentors

Deliverables:

- Agreed technical plan with mentors
- Dev environment ready
- Detailed plan shared with mentors

Phase 1: Core Connector Pipeline

Week	Key Tasks	Deliverables
Week 1 (May 25–31)	- Create `debezium-connector-milvus` module skeleton - Set up CI, Checkstyle, and project structure - Implement `EtcdMetadataReader` for schema, vchannel, and checkpoint bootstrap	- Module builds successfully - etcd bootstrap works in integration test
Week 2 (Jun 1–7)	- Integrate `milvus-proto` codegen into the build - Implement `MilvusProtoDeserializer` for DML, DDL, and timetick message types - Add column-to-row transformation for INSERT payloads	- Deserializer covers all message types - Unit tests pass for message parsing and row reconstruction
Week 3 (Jun 8–14)	- Implement `MilvusSchema` for in-memory schema tracking and Milvus-to-Debezium type conversion - Define `MilvusMessageConsumer` abstraction - Implement `KafkaMilvusMessageConsumer` - Add offset serialization in `MilvusOffsetContext`	- Schema conversion works for all supported field types - Kafka MQ consumption works in integration test - Offset round-trip validated
Week 4 (Jun 15–21)	- Implement `MilvusSnapshotChangeEventSource` - Complete snapshot-to-streaming handoff using etcd checkpoint and `guarantee_ts` - Emit snapshot events through Debezium's event pipeline	- Snapshot works against live Milvus - Streaming start position recorded correctly
Week 5 (Jun 22–28)	- Implement `MilvusStreamingChangeEventSource` - Add `MilvusChangeEventSourceFactory` - Wire connector into Debezium's source pipeline end to end	- First end-to-end prototype working - Milvus change → Debezium event flow validated

Phase 2: Correctness, Recovery, and Stabilization

Week	Key Tasks	Deliverables
Week 6 (Jun 29 – Jul 5)	- Improve restart and recovery behavior - Add DELETE handling and verify DDL/DML ordering through timetick - Add basic retry and error-handling logic for MQ and SDK interactions	- Restart recovery validated - Ordered schema and data events validated - Core failure-handling path implemented
Week 7 (Jul 6–12)	- Test snapshot boundary cases (restart mid-snapshot, empty collections) - Validate handoff correctness under edge cases - Finalize error handling for timetick stall and MQ offset expiry	- Snapshot boundary and restart scenarios covered by integration tests - Timetick stall and offset expiry error handling complete
Week 8 (Jul 13–19)	**Buffer** — address review feedback and outstanding issues from Phase 1–2; prepare and submit midterm report	- Midterm report submitted - Core pipeline stable and review-ready

Phase 3: Schema Handling and Documentation

Week	Key Tasks	Deliverables
Week 9 (Jul 20–26)	- Improve schema handling for collection create and drop flows - Add connector configuration validation - Expand Kafka integration test matrix: schema evolution, restart under load, offset expiry recovery, empty collection snapshot	- Schema evolution and recovery flows correct in integration tests - Config validation implemented
Week 10 (Jul 27 – Aug 2)	- Measure basic performance baseline - Prepare Docker Compose demo environment - Write user-facing documentation (config, supported field types, limitations)	- Demo environment runnable - Documentation complete - Performance baseline recorded

Week	Key Tasks	Deliverables
Week 11 (Aug 3–16)	**Buffer** — fix remaining issues, address review feedback; write user-facing documentation (config, supported field types, limitations); begin Pulsar stretch work if schedule permits	- All outstanding issues resolved - Documentation complete - Codebase ready for final submission

Final Week (Aug 17–23)

- Final cleanup and testing
- Prepare release notes and final documentation updates
- Fix any last-minute bugs
- Submit final work product and mentor evaluation by Aug 24 deadline
- Publish final writeup on project design decisions and lessons learned

Deliverables:

- Final code and documentation submitted by Aug 24
- Release notes and project summary complete
- Final writeup published

Stretch Goal: If the Kafka-based connector is stable on schedule, extend `MilvusMessageConsumer` with a Pulsar implementation during Week 11. Beyond the GSoC period, I plan to continue maintaining the connector, fixing bugs, and contributing to Milvus 2.6 WAL (Woodpecker) support as it stabilizes.

Other commitments

I do not expect any major external commitments during the GSoC coding period beyond my regular coursework. I will prioritize this project accordingly and will communicate early if any scheduling conflicts arise.

Appendix

The following edge cases are handled within `MilvusStreamingChangeEventSource` (see Component 7) and `MilvusSnapshotChangeEventSource` (see Component 6):

- **Drop collection with buffered DML** (Case 5): If a `DropCollectionMsg` is received while DML events for that collection are still buffered, all buffered DML events are discarded, `MilvusSchema.removeSchema()` is called, and a drop schema change event is emitted.
- **Timetick interruption** (Case 6): If no `TimeTickMsg` is received within a configurable timeout, buffered events are force-flushed. If the stall persists beyond a second threshold, the connector stops and waits for manual intervention.
- **MQ offset expired** (Case 7): If the stored MQ checkpoint position is no longer available on the broker, the connector stops with a descriptive error and waits for manual intervention.
- **Collection not found at startup** (Case 8): If `EtcdMetadataReader` cannot find metadata for a configured collection in etcd, the connector stops immediately following Debezium's fail-fast convention.